

Once we have established a bit field, we can reference it just like any other structure element, as shown in the program given below:

```
#include <stdio.h>
#define MALE 0;
#define FEMALE 1;
#define SINGLE 0;
#define MARRIED 1;
#define DIVORCED 2;
#define WIDOWED 3;

void main( )
{
    struct employee
    {
        unsigned gender : 1;
        unsigned mar_stat : 2;
        unsigned hobby : 3;
        unsigned scheme : 4;
    };
    struct employee e;

    e.gender = MALE;
    e.mar_status = DIVORCED;
    e.hobby = 5;
    e.scheme = 9;

    printf ( "\nGender = %d", e.gender );
    printf ( "\nMarital status = %d", e.mar_status );
    printf ( "\nBytes occupied by e = %d", sizeof ( e ) );
}
```

And here is the output...

```
Gender = 0
Marital status = 2
Bytes occupied by e = 2
```

Pointers to Functions

Every type of variable that we have discussed so far, with the exception of register, has an address. We have seen how we can reference variables of the type **char**, **int**, **float**, etc. through their addresses—that is by using pointers. Pointers can also point to C functions. And why not? C functions have addresses. If we know the function's address, we can point to it, which provides another way to invoke it. Let us see how this can be done.

```
#include <stdio.h>
void display( );
void main( )
{
    printf ( "\nAddress of function display is %u", display );
    display( ); /* usual way of invoking a function */
}

void display( )
{
    puts ( "\nLong live viruses!!" );
}
```

The output of the program would be:

```
Address of function display is 1125
Long live viruses!!
```

Note that, to obtain the address of a function, all that we have to do is mention the name of the function, as has been done in the **printf()** statement above. This is similar to mentioning the name of the array to get its base address.

Now let us see how using the address of a function, we can manage to invoke it. This is shown in the program given below:

```
/* Invoking a function using a pointer to a function */
```